Inside the Spec

Realistic Scenarios. Working Artifacts.
Decisions That Matter.

The API timed out. Did the payment succeed?

One top-up. An unknown outcome.
A specification for what happens next.

CASE 01 · PAYMENT TIMEOUT

€50. No reliable answer.

A user tops up a balance by €50. The provider may accept the payment before the request times out.

No status response and no operation ID? Replay POST with the original key within the client budget. Once a status response or ID arrives, “Try again” becomes “Check status” using GET.

> **A timeout does not prove payment failure.**

Case assumptions: one provider; 24-hour concurrent idempotency; stable-reference lookup; immutable final outcomes; an instrument requiring no user interaction. €50 implies no EU card or SCA exemption. Verify the real contract.

Three related state dimensions.

payment_status	SUCCEEDED	Verified payment success.
credit_status	REVIEW_REQUIRED	No authorized attempt; credit blocked.
review_status	OPEN	SECURITY_HOLD is open and blocks new credit dispatch.

Review items are the source of truth. review_status and blocking_review_reasons are derived; credit_status records progress and evidence. Open review does not always mean a credit hold.

Allowed combinations, explicit limits.

Full credit_status enum: NOT_REQUESTED, PENDING, DISPATCHING, OUTCOME_UNKNOWN, REVIEW_REQUIRED, POSTED.

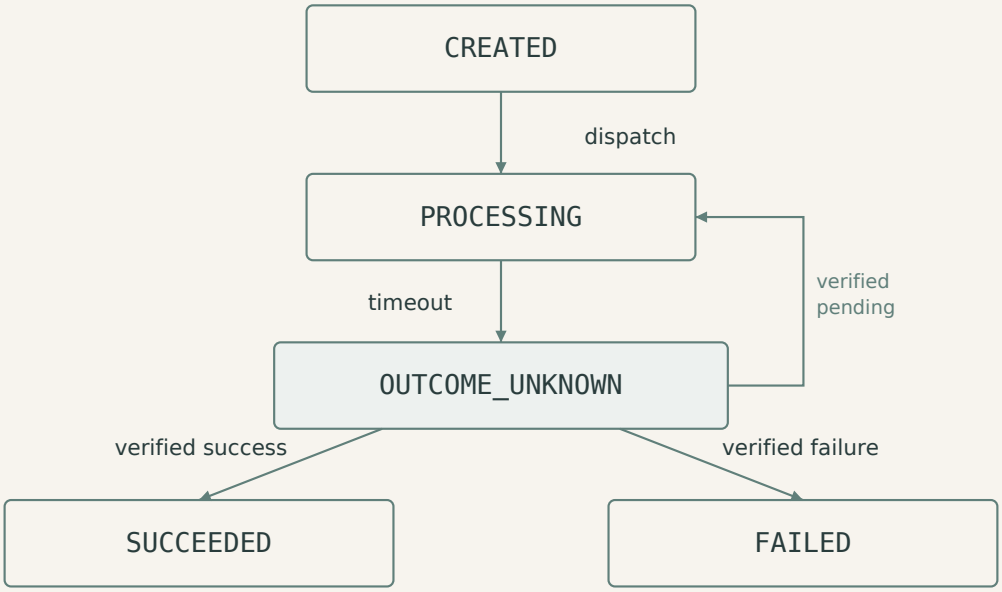
Payment / credit	Constraint
Nonfinal / NOT_REQUESTED	No credit before payment success.
FAILED / NOT_REQUESTED	FAILED + POSTED is forbidden.
SUCCEEDED / PENDING	No blocker or unresolved delivery.
SUCCEEDED / REVIEW_REQUIRED	Blocker open; no authorized attempt can still post.
SUCCEEDED / DISPATCHING	Authorized attempt active; later hold may coexist.
SUCCEEDED / OUTCOME_UNKNOWN	Attempt may have posted; hold may coexist.
SUCCEEDED / POSTED	Posting verified, even with an open review.

> **RESULT_OVERDUE: nonblocking. SECURITY_HOLD: blocks new credit dispatch.**

All other payment/credit combinations are forbidden. OPEN + PENDING is allowed only for nonblocking reviews. A hold never erases DISPATCHING, unknown credit, or POSTED.

Keep the unknown visible.

The diagram follows recovery after an ambiguous create request. State names describe what our service knows.



Also allowed: PROCESSING → either verified final outcome, and CREATED → FAILED for a definitive local rejection before dispatch. Full transition rules are in the specification.

> **Neither NOT_FOUND nor an expired window means FAILED.**

Once any verified provider PENDING is seen, persist provider_operation_seen=true. Never authorize another CREATE, even after a later NOT_FOUND.

Link success, review, and credit.

Trigger	Atomic local result
Verified payment success	SUCCEEDED + credit gate in one payment-DB transaction.
No credit blocker	PENDING + CreditRequested(payment_id)
Blocking reason open	REVIEW_REQUIRED
Dispatch authorization	DISPATCHING + delivery attempt
Ambiguous delivery	credit_status = OUTCOME_UNKNOWN
Verified balance posting	POSTED

A later balance-DB transaction creates the posting. Repeated success does not reset credit progress. A hold after authorization stops new attempts, not the authorized one.

ID legend: PA = provider assumption; ST = payment transition; IC/IM = idempotency rule/matrix; CR = credit rule; AC = acceptance scenario; RC = reconciliation. M, WR, RP, WH, KEY identify model, worker, recovery-clock, webhook, and key policies.

One intent. Stable identity.

Identifier	What it identifies
client_idempotency_key	One user intent. Retain the binding for the operation lifetime; no expiry while unresolved.
payment_id	One internal payment operation.
provider_idempotency_key	The same provider operation across create retries.
attempt_id	One outgoing CREATE or LOOKUP call.

> **Same client key + same parameters → return the existing operation.**

Key scope: user + operation type + client key. The fingerprint covers normalized amount, currency, and instrument.

Save the key, fingerprint, and operation atomically. A storage constraint enforces uniqueness.

Specify the repeated request.

Condition	API behavior
Same key, same data; any nonfinal state	202 · Full PaymentStatus DTO. Continue with GET.
Same key, changed data	409 · IDEMPOTENCY_KEY_REUSED.
Same key, final payment	200 · Same outcome; report credit state separately.
New key; matching unresolved payment	409 · Existing PaymentStatus + SHOW_EXISTING_PAYMENT.
Cannot save operation reliably	503 · No external call without confirmed durable storage.

> **A client retry does not itself trigger another provider create.**

Case HTTP mapping: changed data → 409; in-progress replay → 202. IETF draft-07 recommends 422 / 409 respectively. Retention: ≥30 days from creation and final complete resolution, and the whole operation lifetime.

202 changes the button to GET.

POST returns 202 for every nonfinal payment, 200 for a final one. This DTO fragment omits credit/review fields shown on the previous pages.

```
payment_id: pay-001
payment_status: PROCESSING
status_url: /payments/pay-001
blocking_review_reasons: []
next_action: WAIT
poll_after_seconds: 5
retry_post: false
```

> **Neither status response nor ID: at most one automatic same-key POST replay. Status response or known ID: “Check status” with GET only.**

GET returns 200 for known nonfinal operations too. Poll at most once per 5 seconds, for at most 60 seconds; then retain manual refresh. A duplicate guard returns the full existing operation and SHOW_EXISTING_PAYMENT.

Blocking review or operational handoff → CONTACT_SUPPORT. Nonblocking OPEN alone does not mean held. HTTP 200 does not establish payment or credit success.

One clock, bounded create.

```
T0 = persisted recovery_started_at
LOOKUP at T0 + 15s / 60s / 300s / 900s / 3600s
CREATE / LOOKUP timeout: 10s
CREATE budget: initial + 1 retry
```

Offsets are absolute, not intervals between calls. At T0+3600s, an independent deadline opens review and stops CREATE before the last lookup starts. Restart: one current check, no burst.

> **Primary safety after an ambiguous CREATE: the same provider key and concurrent idempotency. NOT_FOUND is only a retry gate.**

Lookup-free retry requires all previous attempts proven not accepted. Every route checks budget, blockers, provider_operation_seen=false, and deadlines. The 23-hour cutoff is a defensive outer bound; the one-hour limit normally wins.

Workers resume durable work.

Saved state	Owner / action
CREATED	Dispatch job committed with payment. Watchdog scans every 5s; lease 30s.
CREATED, age ≥60s, no reservation	Watchdog expires unstarted job under lock; FAILED, guard released, key retained.
PROCESSING, worker lost	Watchdog preserves budget, marks unknown, schedules lookup. No blind CREATE.

A stale worker cannot reserve after local expiry. Any reserved attempt is possibly sent. Payments platform on-call owns queue health; Payment Operations owns overdue results. Deadlines require available workers/storage.

An HTTP code is not proof.

Provider response	Recovery decision
429; proven nonacceptance	Honor Retry-After. Retry only if earlier attempts are also proven not accepted.
429; no nonacceptance guarantee	Outcome unknown. Lookup before an eligible retry.
502 / 503 / 504 or timeout	Unknown without a separate contractual guarantee.
Documented definitive rejection	FAILED only when the contract proves final failure.

Proven nonacceptance of attempt 2 does not resolve an unknown attempt 1. Never infer the payment result from a failed lookup.

Close unknown with evidence.

Evidence	Decision
Verified final success / failure	Apply normal final transition; credit gate still applies.
Missing row / NOT_FOUND	Remain unknown. Keep guard and owner.
Final NOT_EXECUTED under PA-08	FAILED only if all attempts are covered and future execution is fenced.

> **Daily reconciliation: 02:00 UTC. Escalate at 24 hours; incident at 7 days. Age never creates a financial result.**

PA-08 requires final nonexecution evidence covering every attempt and preventing future execution. It is an additional provider capability. Missing rows never prove failure.

A new key is still a repeat risk.

> **Same user + instrument + amount + currency; unresolved payment → 409 with its full status and SHOW_EXISTING_PAYMENT.**

Check an accepted client key first. Serialize new-key guard, creation, and resolution. At 6 minutes or 24 hours, unknown is still blocked. Six minutes is an age example, not a timeout.

Never-dispatched CREATED can expire under WR-01. Sent or ambiguous payments need final evidence. Different instruments/amounts and new intents after final success need separate policy.

Review has reasons, not one flag.

review_status aggregates separate review items. Close one reason without erasing the others.

Late verified success arrives	Expected behavior
Only RESULT_OVERDUE is open	Set SUCCEEDED; close overdue; schedule credit.
Overdue + SECURITY_HOLD	Set SUCCEEDED; close only overdue; hold credit.
A final failure already exists	Keep the recorded final state; open a conflict review.

Review aggregate: NONE → OPEN → RESOLVED; new evidence may reopen it. Every item has an owner, next action, evidence, and an audit trail.

Persist trusted evidence, including rejects.

```
HMAC-SHA256(timestamp + "." + raw_body)
Delivery age: ±300s; body limit: 64 KiB
Business dedup: provider + event_id
```

Condition	Response / evidence
Invalid signature / unknown key	401; security metadata/hash, no business update.
Valid signature, stale timestamp	401 after authenticated quarantine; trusted lookup.
Valid signature, missing event_id	400 after quarantine; no application; trusted lookup.
Trusted amount/currency mismatch	2xx after quarantine; no result applied; correlated review.
Valid event / storage failure	2xx after durable inbox / 503.

Verify original bytes and compare decoded signatures in constant time. Quarantine is encrypted and separate from the business inbox. A body hash deduplicates quarantine receipts; it does not replace event_id.

Secret manager; key-ID rotation with previous key accepted ≤24h. Compromise: revoke immediately, recover via lookup. Retain inbox/quarantine ≥30 days and until linked reviews/operations close.

Credit can be unknown too.

Success plus no blocker creates PENDING and an outbox task locally. Authorization commits DISPATCHING under the shared payment lock. Balance-side atomic deduplication enforces one posting per payment_id, including concurrent delivery.

> **Lost response, then hold: OUTCOME_UNKNOWN stays unknown. Pause new writes; continue read-only lookup by payment_id.**

A verified posting becomes POSTED even during hold. NOT_FOUND does not prove no credit: an old authorization may still execute. REVIEW_REQUIRED needs a blocker and no possible earlier effect.

Without a hold, redeliver the same task/payment_id safely. While held, do not retry an idempotent write that could create a posting. Strict revocation after authorization needs a balance-owned protocol.

Reconcile and keep delivery moving.

Balance Recovery owns lookup: +5s after ambiguity, every 30s to 2m, then every 5m. Check posting ID, user, amount, and currency. Balance Operations takes unposted credit at success +24h; tasks remain durable.

Write/lookup timeout 10s; shared 10 calls/s and 10 concurrent. Backoff with jitter and circuit breaker; retain tasks. SLO target: 99.9% in 60s, warning at 2m from payment success; unmeasured.

Eventually exactly one posting requires eventual hold resolution, valid data, retained tasks, fair worker processing, and restored balance/storage availability. Never more than one posting is the safety rule. Full SLO denominator and exclusions: specification.

A queue read is not authorization.

Before dispatch, lock the payment row, reread holds, and commit DISPATCH_AUTHORIZED. Hold writers lock the same row.

Commit order	Consequence
Hold before authorization	No balance call, even if the worker already read the task.
Authorization before hold	That attempt may execute. Hold stops later attempts.
First response lost, no hold	Redeliver same payment_id; return or create one posting.
Response lost, then hold	OUTCOME_UNKNOWN; read-only lookup continues, new writes stop.

> **Outbox atomicity is local. The balance DB commits separately and enforces posting uniqueness.**

Authorization commit is the logical start boundary. A later hold cannot strictly revoke this call. Strict revocation needs a balance-owned protocol, not another local read.

The second writer must reread.

Concurrent evidence	Required result
Lookup PENDING + success webhook	SUCCEEDED; old pending cannot downgrade it.
Success from both channels	One transition, one logical credit task, one posting.
Final success + final failure	Retain first accepted final state; open conflict.
Timeout + verified success	Retain success regardless of commit order.

Persist evidence. Check the operation version. Commit the transition and outbox atomically. On conflict, reload and reevaluate the same evidence.

Local locks cannot cancel an in-flight request. Overlapping provider calls also require the provider’s concurrent idempotency guarantee.

Test the uncomfortable cases.

Scenario	Acceptance result
AC-03 · Both create calls time out	Unknown remains; no third create.
AC-09 · Late success during security hold	Success recorded; only overdue closes; credit held.
AC-18 · Posting exists, response lost	Redelivery returns the same posting.
AC-22 · Different keys arrive together	Guard accepts one operation; the other gets 409.
AC-43 · Lost credit response + hold	Unknown credit, no new write, lookup remains active.

> **58 acceptance scenarios in the specification. AC IDs link each expected result to its rules.**

These are expected outcomes, not executed implementation tests. Provider assumptions, concurrency, and the SLO still require validation against a real system.

Beyond the Scope

Process	What this case does not specify
3DS; authorization / capture	User action, authorization, and capture lifecycles.
Refund; reversal; chargeback	Separate financial operations after payment success.
Provider failover; partials; FX	Cross-provider duplicates, remaining amounts, and conversion.
Provider report adapters	Report formats and evidence of finality.

Also separate: correcting final outcomes, strict balance-side revocation, and duplicates across instruments or after final success.

Full specification – see GitHub.

GitHub Repository for the Inside the Spec

Yulia Melekhova on LinkedIn